

CS3014 - Estructura de Datos Avanzados

Retroactividad

Semana 3

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

28 de agosto de 2026

¿Qué aprenderemos hoy?



¿Qué aprenderemos hoy?



- ▶ El concepto de retroactividad (y en qué se diferencia de la persistencia)

¿Qué aprenderemos hoy?



- ▶ El concepto de retroactividad (y en qué se diferencia de la persistencia)
- ▶ El caso fácil: operaciones conmutativas e invertibles

¿Qué aprenderemos hoy?



- ▶ El concepto de retroactividad (y en qué se diferencia de la persistencia)
- ▶ El caso fácil: operaciones conmutativas e invertibles
- ▶ Problemas de búsqueda descomponibles, vía Segment Tree

¿Qué aprenderemos hoy?



- ▶ El concepto de retroactividad (y en qué se diferencia de la persistencia)
- ▶ El caso fácil: operaciones conmutativas e invertibles
- ▶ Problemas de búsqueda descomponibles, vía Segment Tree
- ▶ El método general de *rollback*, y su costo

¿Qué aprenderemos hoy?



- ▶ El concepto de retroactividad (y en qué se diferencia de la persistencia)
- ▶ El caso fácil: operaciones conmutativas e invertibles
- ▶ Problemas de búsqueda descomponibles, vía Segment Tree
- ▶ El método general de *rollback*, y su costo
- ▶ Caso de estudio: Priority Queue retroactiva

El Concepto de Retroactividad

Motivación: ¿y si me equivoco en el pasado?



El escenario

Ejecutaste una secuencia larga de operaciones sobre una estructura de datos. Ahora te das cuenta de que **olvidaste una operación** en algún punto **del pasado** — o que una operación pasada estuvo **mal** y hay que borrarla.

Motivación: ¿y si me equivoco en el pasado?



El escenario

Ejecutaste una secuencia larga de operaciones sobre una estructura de datos. Ahora te das cuenta de que **olvidaste una operación** en algún punto **del pasado** — o que una operación pasada estuvo **mal** y hay que borrarla.

¡Piénsalo!

La solución ingenua es rehacer todo desde ese punto. Si han pasado m operaciones desde entonces, eso cuesta $O(m)$. ¿Se puede hacer mejor?

Retroactividad vs. persistencia



Persistencia

Al modificar el **pasado**, se crea una **versión nueva**; la versión “presente” original sigue existiendo, intacta, en paralelo.

Retroactividad vs. persistencia



Persistencia

Al modificar el **pasado**, se crea una **versión nueva**; la versión “presente” original sigue existiendo, intacta, en paralelo.

Retroactividad

Al modificar el **pasado**, el **presente** se **actualiza** para reflejar ese cambio — como si el cambio siempre hubiera estado ahí. Solo existe una línea de tiempo.

Retroactividad vs. persistencia



Persistencia

Al modificar el **pasado**, se crea una **versión nueva**; la versión “presente” original sigue existiendo, intacta, en paralelo.

Retroactividad

Al modificar el **pasado**, el **presente** se **actualiza** para reflejar ese cambio — como si el cambio siempre hubiera estado ahí. Solo existe una línea de tiempo.

Son ideas “duales”: persistencia **ramifica** el futuro; retroactividad **reescribe** el pasado y propaga el efecto hacia el presente.

Operaciones sobre la línea de tiempo



El modelo

Una estructura de datos **retroactiva** mantiene una secuencia de operaciones, cada una con un tiempo asociado, y soporta:

Operaciones sobre la línea de tiempo



El modelo

Una estructura de datos **retroactiva** mantiene una secuencia de operaciones, cada una con un tiempo asociado, y soporta:

- ▶ $\text{INSERT}(t, op)$: inserta la operación op en el tiempo t (que puede ser **pasado**).

Operaciones sobre la línea de tiempo



El modelo

Una estructura de datos **retroactiva** mantiene una secuencia de operaciones, cada una con un tiempo asociado, y soporta:

- ▶ $\text{INSERT}(t, op)$: inserta la operación op en el tiempo t (que puede ser **pasado**).
- ▶ $\text{DELETE}(t)$: elimina la operación que estaba en el tiempo t .

Operaciones sobre la línea de tiempo



El modelo

Una estructura de datos **retroactiva** mantiene una secuencia de operaciones, cada una con un tiempo asociado, y soporta:

- ▶ $\text{INSERT}(t, op)$: inserta la operación op en el tiempo t (que puede ser **pasado**).
- ▶ $\text{DELETE}(t)$: elimina la operación que estaba en el tiempo t .
- ▶ $\text{QUERY}(t, op)$: responde una consulta como si se hiciera en el tiempo t .

Retroactividad parcial vs. completa



Retroactividad parcial

Las **consultas** solo se permiten en el **presente** (el tiempo más reciente); solo las actualizaciones (INSERT/DELETE) pueden apuntar al pasado.

Retroactividad parcial vs. completa



Retroactividad parcial

Las **consultas** solo se permiten en el **presente** (el tiempo más reciente); solo las actualizaciones (INSERT/DELETE) pueden apuntar al pasado.

Retroactividad completa (*full*)

Las **consultas** también se permiten en **cualquier** momento del pasado.

Retroactividad parcial vs. completa



Retroactividad parcial

Las **consultas** solo se permiten en el **presente** (el tiempo más reciente); solo las actualizaciones (INSERT/DELETE) pueden apuntar al pasado.

Retroactividad completa (*full*)

Las **consultas** también se permiten en **cualquier** momento del pasado.

Igual que con persistencia, “parcial” suele ser significativamente más barato de lograr que “completa”.

El Caso Fácil

Actualizaciones conmutativas e invertibles



Conmutativas

op_x seguida de op_y tiene el **mismo efecto** que op_y seguida de op_x : el **orden no importa**.

Actualizaciones conmutativas e invertibles



Conmutativas

op_x seguida de op_y tiene el **mismo efecto** que op_y seguida de op_x : el **orden no importa**.

Invertibles

Existe una operación op_x^{-1} tal que aplicar op_x y luego op_x^{-1} deja la estructura **exactamente** como estaba.

Actualizaciones conmutativas e invertibles



Conmutativas

op_x seguida de op_y tiene el **mismo efecto** que op_y seguida de op_x : el **orden no importa**.

Invertibles

Existe una operación op_x^{-1} tal que aplicar op_x y luego op_x^{-1} deja la estructura **exactamente** como estaba.

Si **ambas** propiedades se cumplen, la retroactividad parcial es casi trivial.

Por qué alcanza con esto



INSERT(t, op) retroactivo

Como op conmuta con todo lo demás, **no importa** en qué posición de la línea de tiempo se aplique realmente:

$$\text{INSERT}(t, op) \equiv \text{INSERT}(\text{ahora}, op).$$

Se aplica **directamente** sobre la estructura actual.

Por qué alcanza con esto



INSERT(t, op) retroactivo

Como op conmuta con todo lo demás, **no importa** en qué posición de la línea de tiempo se aplique realmente:

$$\text{INSERT}(t, op) \equiv \text{INSERT}(\text{ahora}, op).$$

Se aplica **directamente** sobre la estructura actual.

DELETE(t) retroactivo

Si la operación que había en t era op , basta aplicar su inversa **ahora**:

$$\text{DELETE}(t) \equiv \text{INSERT}(\text{ahora}, op^{-1}).$$

Ejemplos



Tabla hash con inserciones

Insertar una llave nueva conmuta con insertar otra llave distinta; eliminarla es la operación inversa. Retroactividad parcial: $O(1)$ amortizado, igual que la operación original.

Ejemplos



Tabla hash con inserciones

Insertar una llave nueva conmuta con insertar otra llave distinta; eliminarla es la operación inversa. Retroactividad parcial: $O(1)$ amortizado, igual que la operación original.

Arreglo con $A[i] \ += \Delta$

Sumar Δ a la posición i conmuta consigo misma en cualquier posición, y su inversa es sumar $-\Delta$. Retroactividad parcial: $O(1)$, igual que la operación original.

Problemas de Búsqueda Descomponibles

¿Qué pasa cuando las operaciones NO conmutan?



El problema

La mayoría de las estructuras interesantes (pilas, colas, árboles de búsqueda, *priority queues*. . .) tienen operaciones que sí dependen del orden. El truco anterior no aplica directamente.

¿Qué pasa cuando las operaciones NO conmutan?



El problema

La mayoría de las estructuras interesantes (pilas, colas, árboles de búsqueda, *priority queues*. . .) tienen operaciones que sí dependen del orden. El truco anterior no aplica directamente.

Necesitamos algo más general.

Problema de búsqueda descomponible



Definición

Un problema de búsqueda sobre un conjunto S es **descomponible** si, para cualquier partición $S = A \cup B$:

$$\text{QUERY}(x, A \cup B) = f(\text{QUERY}(x, A), \text{QUERY}(x, B))$$

para alguna función f calculable en $O(1)$.

Problema de búsqueda descomponible



Definición

Un problema de búsqueda sobre un conjunto S es **descomponible** si, para cualquier partición $S = A \cup B$:

$$\text{QUERY}(x, A \cup B) = f(\text{QUERY}(x, A), \text{QUERY}(x, B))$$

para alguna función f calculable en $O(1)$.

Ejemplos

Mínimo, máximo, suma, existencia de un elemento — todos son descomponibles: $f = \text{mín}, \text{máx}, +, \vee$, respectivamente.

La conexión con la clase pasada: Segment Tree



La idea

Construimos un **Segment Tree** sobre el **eje del tiempo**: cada hoja es un instante entre dos operaciones consecutivas, y cada nodo interno resume, con f , el efecto acumulado de **todo su rango de tiempo**.

La conexión con la clase pasada: Segment Tree



La idea

Construimos un **Segment Tree** sobre el **eje del tiempo**: cada hoja es un instante entre dos operaciones consecutivas, y cada nodo interno resume, con f , el efecto acumulado de **todo su rango de tiempo**.

Insertar/eliminar una operación en el tiempo t actualiza $O(\lg m)$ nodos del Segment Tree (donde m es el número de operaciones) — ¡el mismo **UPDATE** que ya conocemos!

Retroactividad completa en $O(\lg m)$



El resultado

Cualquier problema de búsqueda descomponible admite retroactividad **completa** con un **overhead multiplicativo** de $O(\lg m)$ por operación, usando un Segment Tree sobre el tiempo.

Retroactividad completa en $O(\lg m)$



El resultado

Cualquier problema de búsqueda descomponible admite retroactividad **completa** con un **overhead multiplicativo** de $O(\lg m)$ por operación, usando un Segment Tree sobre el tiempo.

Si la estructura original respondía en $O(q)$, la versión retroactiva responde en $O(q \lg m)$.

El Método General de Rollback

El método de *rollback*: idea



Cuando nada de lo anterior aplica

Guardamos la secuencia completa de operaciones (un **log**). Para $\text{INSERT}(t, op)$ o $\text{DELETE}(t)$: deshacemos (*rollback*) todas las operaciones posteriores a t , aplicamos el cambio, y las **rehacemos** en orden.

El método de *rollback*: idea



Cuando nada de lo anterior aplica

Guardamos la secuencia completa de operaciones (un **log**). Para $\text{INSERT}(t, op)$ o $\text{DELETE}(t)$: deshacemos (*rollback*) todas las operaciones posteriores a t , aplicamos el cambio, y las **rehacemos** en orden.

Requiere

Que cada operación (y su inversa) sea $O(1)$ u $O(\lg n)$ — cualquier estructura eficiente y **reversible** sirve.

Análisis: por qué cuesta $O(r)$



Costo

Si r es el número de operaciones **después** de t (las que hay que deshacer y rehacer), el costo de un $\text{INSERT}(t, \cdot)$ o $\text{DELETE}(t)$ es

$$O(r) \times (\text{costo de una operación individual}).$$

Análisis: por qué cuesta $O(r)$



Costo

Si r es el número de operaciones **después** de t (las que hay que deshacer y rehacer), el costo de un $\text{INSERT}(t, \cdot)$ o $\text{DELETE}(t)$ es

$$O(r) \times (\text{costo de una operación individual}).$$

Si t está muy **cerca del presente**, r es pequeño y el método es barato; si t está muy **atrás**, $r \approx m$ y es tan caro como rehacerlo todo.

Cota inferior: $\Omega(r)$ es necesario



Un resultado negativo

En general, **no** se puede evitar el factor r : existen estructuras (y secuencias de operaciones) donde modificar el tiempo t **necesariamente** cambia el resultado de consultas que dependen de las r operaciones posteriores, así que **alguna** estructura debe reflejar ese cambio en $\Omega(r)$ de sus partes.

Cota inferior: $\Omega(r)$ es necesario



Un resultado negativo

En general, **no** se puede evitar el factor r : existen estructuras (y secuencias de operaciones) donde modificar el tiempo t **necesariamente** cambia el resultado de consultas que dependen de las r operaciones posteriores, así que **alguna** estructura debe reflejar ese cambio en $\Omega(r)$ de sus partes.

Por eso las técnicas especializadas (como la de la siguiente sección) son valiosas: rompen esta barrera aprovechando propiedades específicas del problema.

Caso de Estudio: Priority Queue

El problema concreto



Priority Queue (montículo)

Soporta $\text{INSERT}(k)$ y DELETE-MIN , cada una en $O(\lg n)$. Queremos volverla **retroactiva parcial**: poder insertar o eliminar operaciones en el **pasado** de la línea de tiempo, y consultar el estado **actual**.

El problema concreto



Priority Queue (montículo)

Soporta $\text{INSERT}(k)$ y DELETE-MIN , cada una en $O(\lg n)$. Queremos volverla **retroactiva parcial**: poder insertar o eliminar operaciones en el **pasado** de la línea de tiempo, y consultar el estado **actual**.

Meta: lograrlo en $O(\lg n)$ **amortizado** por operación retroactiva — mucho mejor que el rollback genérico, y mejor incluso que el $O(\lg n \cdot \lg m)$ que daría el Segment Tree genérico (DELETE-MIN no es un problema descomponible simple).

¿Por qué es difícil?



El problema de fondo

DELETE-MIN “recuerda” implícitamente **todo** lo insertado antes: elimina el mínimo **de entre todo lo que sigue vivo**. Insertar una llave nueva en el pasado puede cambiar **qué** elemento eliminó cada DELETE-MIN posterior — en cascada, uno tras otro.

¿Por qué es difícil?



El problema de fondo

DELETE-MIN “recuerda” implícitamente **todo** lo insertado antes: elimina el mínimo **de entre todo lo que sigue vivo**. Insertar una llave nueva en el pasado puede cambiar **qué** elemento eliminó cada DELETE-MIN posterior — en cascada, uno tras otro.

¡Piénsalo!

Si insertas k en el pasado y k es menor que lo que se eliminó en el siguiente DELETE-MIN, ese elemento “sobrevive” un turno más. . . ¿y entonces qué pasa con el **siguiente** DELETE-MIN?

La idea clave: el efecto neto es simple



El resultado sorprendente

A pesar de la cascada aparente, insertar k en el tiempo t tiene, en el **presente**, un efecto neto muy simple: o bien k termina en Q_{ahora} , o bien **reemplaza** (en el conjunto de elementos eliminados) a exactamente **un** elemento — ¡nunca más que eso!

La idea clave: el efecto neto es simple



El resultado sorprendente

A pesar de la cascada aparente, insertar k en el tiempo t tiene, en el **presente**, un efecto neto muy simple: o bien k termina en Q_{ahora} , o bien **reemplaza** (en el conjunto de elementos eliminados) a exactamente **un** elemento — ¡nunca más que eso!

La cascada de “quién se elimina en cada paso” se resuelve, al llegar al presente, en un solo intercambio.

Formalizando el efecto neto



INSERT(t , 'insert(k)') retroactivo

Sea

$$M = \text{máx} (\{k\} \cup \{k' : k' \text{ fue eliminado en tiempo } \geq t\}).$$

Formalizando el efecto neto



INSERT(t , 'insert(k)') retroactivo

Sea

$$M = \max(\{k\} \cup \{k' : k' \text{ fue eliminado en tiempo } \geq t\}).$$

Efecto neto sobre Q_{ahora}

Se inserta M en Q_{ahora} .

- ▶ Si $M = k$: k era “demasiado grande” para haber sido eliminado por ningún DELETE-MIN en el camino — simplemente se agrega a Q_{ahora} .
- ▶ Si $M \neq k$: M es el mayor de los eliminados desde t ; “sobrevive” hasta ahora y entra a Q_{ahora} , mientras k toma su lugar entre los eliminados.

El costo de calcular M : el problema



Calculado ingenuamente

Encontrar el máximo entre **todos** los elementos eliminados desde el tiempo t hasta ahora requeriría revisar, en el peor caso, las r operaciones posteriores a t — de vuelta al $O(r)$ del *rollback* genérico.

El costo de calcular M : el problema



Calculado ingenuamente

Encontrar el máximo entre **todos** los elementos eliminados desde el tiempo t hasta ahora requeriría revisar, en el peor caso, las r operaciones posteriores a t — de vuelta al $O(r)$ del *rollback* genérico.

Necesitamos una forma de acotar la búsqueda

La solución: identificar puntos “seguros” en la línea de tiempo que acoten cuánto hay que mirar hacia atrás.

El concepto de *bridge* (puente)



Definición

Un tiempo t' es un **puente** si $Q_{t'} \subseteq Q_{\text{ahora}}$: es decir, **todo** elemento presente en la *priority queue* en el instante t' **sigue** presente en el presente (nada de lo que había en t' fue eliminado después).

El concepto de *bridge* (puente)



Definición

Un tiempo t' es un **puente** si $Q_{t'} \subseteq Q_{\text{ahora}}$: es decir, **todo** elemento presente en la *priority queue* en el instante t' **sigue** presente en el presente (nada de lo que había en t' fue eliminado después).

¿Por qué esto ayuda?

Si t' es un puente, ningún elemento presente en t' puede aparecer en el conjunto “eliminado desde t ” para ningún $t \geq t'$ — son, por definición, sobrevivientes.

La fórmula final, usando el puente más cercano



Sea t' el puente más cercano antes de t

$$\text{máx}\{k' : k' \text{ eliminado en tiempo} \geq t\}$$

$$\text{máx}\{k' \notin Q_{\text{ahora}} : k' \text{ insertado en tiempo} \geq t'\}.$$

La fórmula final, usando el puente más cercano

Sea t' el puente más cercano **antes** de t

$$\text{máx}\{k' : k' \text{ eliminado en tiempo} \geq t\}$$

$$\text{máx}\{k' \notin Q_{\text{ahora}} : k' \text{ insertado en tiempo} \geq t'\}.$$

La búsqueda se reduce a mirar solo lo insertado **después del puente**, no toda la historia — y se mantiene con una estructura balanceada (BST) indexada por tiempo, con información agregada (máximo de lo no presente en Q_{ahora}) en cada subárbol.

Ejemplo trabajado



tiempo	1	2	3	4	5
operación	ins(5)	ins(2)	del-min	ins(8)	del-min
efecto	{5}	{5, 2}	elimina 2	{5, 8}	elimina 5

Ejemplo trabajado

tiempo	1	2	3	4	5
operación	ins(5)	ins(2)	del-min	ins(8)	del-min
efecto	{5}	{5, 2}	elimina 2	{5, 8}	elimina 5

$Q_{\text{ahora}} = \{8\}$; los eliminados desde $t = 2$: $\{2, 5\}$

Insertamos retroactivamente INSERT(2,5, 'insert(3)') (entre los tiempos 2 y 3).

Ejemplo trabajado

tiempo	1	2	3	4	5
operación	ins(5)	ins(2)	del-min	ins(8)	del-min
efecto	{5}	{5, 2}	elimina 2	{5, 8}	elimina 5

$Q_{\text{ahora}} = \{8\}$; los eliminados desde $t = 2$: $\{2, 5\}$

Insertamos retroactivamente INSERT(2,5, 'insert(3)') (entre los tiempos 2 y 3).

Cálculo

$M = \max(\{3\} \cup \{2, 5\}) = 5$. Como $M \neq 3$: se inserta 5 en Q_{ahora} (que pasa a ser $\{8, 5\}$), y 3 pasa a formar parte del conjunto de eliminados. Justo lo que esperaríamos: si 3 se hubiera insertado antes del primer DELETE-MIN, ese DELETE-MIN habría eliminado 2 igual (es el menor), pero el segundo DELETE-MIN ahora elimina 3 en vez de 5 — y 5 sobrevive.

Complejidad final



El resultado (Demaine, Iacono, Langerman, 2007)

Con la estructura de puentes mantenida sobre un árbol balanceado, INSERT y DELETE retroactivos parciales sobre una *priority queue* cuestan $O(\lg n)$ **amortizado** — el mismo orden que las operaciones originales.

Complejidad final



El resultado (Demaine, Iacono, Langerman, 2007)

Con la estructura de puentes mantenida sobre un árbol balanceado, INSERT y DELETE retroactivos parciales sobre una *priority queue* cuestan $O(\lg n)$ **amortizado** — el mismo orden que las operaciones originales.

Un ejemplo de cómo entender la **estructura** del problema (aquí, que el efecto de una inserción retroactiva se reduce a un único intercambio) permite superar por mucho la cota genérica $O(r)$ del *rollback*.

Resumen de la sesión



- ▶ La **retroactividad** actualiza el presente al modificar el pasado (a diferencia de la persistencia, que ramifica el futuro).
- ▶ Con operaciones **conmutativas e invertibles**, la retroactividad parcial es casi gratis.
- ▶ Para **problemas de búsqueda descomponibles**, un Segment Tree sobre el tiempo da retroactividad completa con overhead $O(\lg m)$.
- ▶ El método genérico de **rollback** cuesta $O(r)$, y esto es **necesario** en general ($\Omega(r)$).
- ▶ La *priority queue* retroactiva parcial logra $O(\lg n)$ amortizado explotando que insertar en el pasado solo produce **un** intercambio neto en el presente, localizable con **puentes**.

Gracias

Referencias



Bibliografía citada

- ▶ Demaine, E. D., Iacono, J., & Langerman, S. (2007). Retroactive data structures. *ACM Transactions on Algorithms*, 3(2).
- ▶ Cormen, Thomas H. et al. (2022). *Introduction to Algorithms*. 4th ed. The MIT Press.
- ▶ Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. <https://courses.csail.mit.edu/6.851/spring21/>.

Agradecimientos



Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.

Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.